Serial Protocol Decode — Keithley DMM6500, DAQ6510, DMM7510

A UART decoder that runs **on** a Keithley bench instrument. It digitizes the line with the instrument's own digitizer, recovers the baud rate, frame format and idle polarity from the signal, and shows the bytes on the front panel as text or hex. No host, no logic analyser, one probe.

Tested on a DMM6500 and nothing else — every measured number in these documents came off that one instrument. What it should run on unmodified is the wider TSP family; see **Which instruments** below for what is claim and what is measurement.

Ian Ameline · **version 1.05 — beta** · MIT licence (see LICENSE) · user manual

Beta status. Everything the manual claims was measured on the bench, and the release gate below is what measures it — it must pass with zero failures before a build goes out. One rough edge worth naming here: **flow control** is verified electrically — one credit pulse per armed capture, measured idle-low at **+0.20 V, +4.45 V high, 5.72 µs wide with a 436 ns rise** — but has never been closed as a loop against a device that actually waits for it, so the "unlimited with flow control" claim is sound by construction rather than demonstrated. The pulse width is **not settable** on this firmware — `trigger.extout.pulsewidth` is nil on 1.7.17a — so a device waiting for a credit needs an edge-triggered input rather than a polling loop. What *has* been located since is where the loop fails: the generator's burst machinery accepts the arbs, and it did not act on this pulse. See **Triggering, in both directions** in docs/REFERENCE.md.

Automatic rate detection has two known failures, both rare and both fixable by typing the rate in — which is what most decoders require of you anyway. A short pattern repeated over and over — `00` / `FF` / `55` / `AA`, walking bits, a fixed string on a loop — can be measured at a small multiple of its true rate: **24 of 6,714** bench points across four full sweeps, **0.36 %**, every one a repeating pattern at a non-standard rate. Offline the split is stark — **0 of 6,314 standard-rate points** against 24 of 6,027 non-standard ones — but on the instrument a waveform carrying a **LIN-style break field**, which is not valid 8N1 at all, *was* measured at three times its true rate on standard rates, silently. No ordinary UART payload has shown it at any standard rate. Separately, a line whose logic low sits around **–2 V** can misdetect, while –0.1, –0.5, –1.0 and –3.0 V all measure clean. Both are characterised with their numbers under **Known failures of automatic rate detection** in the manual.

Endurance, measured — and reproduced. Two independent 17-hour soaks, each **6 laps of the full 1,683-point matrix, 10,098 capture-and-decode cycles on one power cycle**, with **no instrument event logged, no buffer or display object leaked and no error left behind** in either. They were run on different cabling and with a change to the app's state handling between them, and they agree:

	duration	failures of 10,098	drift
first	17.14 h	318	–0.9 %
second	17.17 h	313	–1.5 %

Within 0.2 % on duration and 1.6 % on failure count, across 20,196 capture-and-decode cycles. Two runs that reproduce each other are worth much more than one that passed, because a single run cannot distinguish a stable failure rate from a lucky evening.

The number worth more than any pass count is still the **lap time**, and in both runs it *fell* — the second's six laps ran 10 567, 10 219, 10 254, 10 282, 10 173 and 10 299 s, every one within ±1 % of its counterpart in the first. That is the measurement that matters, because a leaked display object, a buffer handle never returned or a fragmenting allocator would not fail a lap: it would wedge the instrument around hour six.

A third, shorter soak is the zero-failure run. Over 7 hours it took **81 laps — 3,726 capture-and-decode cycles and 8 one-press recordings — with zero failures**, every recording ending because its buffer filled at 8 192 bytes. It is the only evidence covering the **recording** path, which neither long soak exercises.

The long soaks are deliberately hostile, and 3.1 % of their points fail — 318 of 10,098 in one and 313 in the other, which is **the same rate arrived at twice**. That number is not comparable to the rate-detection figure above: this matrix exists to attack the decoder, and it includes LIN traffic presented to a UART decoder, single-byte and walking-bit patterns, sinusoidal drift, and swings and offsets swept to the edge of the range.

Two things about the shape of that 3.1 %, because the shape matters more than the number. **It concentrates**: four vectors — `v94`, `v63`, `v90`, `v71` — account for **76 %** of every failure, and `v63` alone is a LIN frame whose 13-bit break field is not valid 8N1 at all. And **it is mostly intermittent**: those 318 failures are only **162 distinct cells**, of which **88 failed in exactly one lap of the six** and just **6 failed in all six**. Most of what fails, fails occasionally.

Which is the whole argument for soaking rather than sweeping. One green sweep would have licensed those 88; one red sweep would have reported a regression nobody could reproduce. Six laps is what turns "it failed" into a rate — and it is why a cell that fails in all six, `v63` at five standard rates and `v94` at 1200, is a property of the cell rather than of the evening.

The second soak is what shows that reasoning holds. Comparing which fixed-stimulus cells failed, **a lap of the second run differs from its counterpart in the first by less than two laps of the first run differ from each other** — 11 to 16 cells against 21 to 32 for every pair of first-run laps. The two runs are closer to each other than one run is to itself, which is the strongest statement available here: the failure set is a property of the matrix, not of the run.

What the soaks do **not** cover: one power cycle rather than many, and no front-panel interaction — that is `hw-panel`'s job.

Nothing stops a long job early — there is no working stop control. A touch press cannot be delivered while a script runs, so the panel cannot offer a stop control, and the front-panel TRIGGER key does not fill the gap: it does **not** deliver `trigger.EVENT_DISPLAY` while a panel-initiated run is executing. So a recording runs for its stated time and cannot be interrupted; decide the size before pressing. The user manual says so under **The TRIGGER key**. The same key does still *gate* an armed capture when `Options ▸ Trigger = Trigger key`, which took its own purpose-built test (`tools/bench_trigkey.py --quiet-line`) because timing-based tests could not tell a prompt press from a free-running capture.

How much it captures, and how many presses. A screenful is ~240 bytes; a recording holds **8 192 or 32 768 bytes** to a file, and **one press** records it, decodes all of it and files it — no stepping. Beyond one window, flow control makes the window a chunk size rather than a total: one press then credits the device, records, decodes and credits again until it stops sending, so an unlimited amount can be captured losslessly **with no interaction per window** by a device built to wait for the credit — bounded at 32 windows or 20 minutes per press, then Mode and Capture resume the same conversation in the next file, with the sender still waiting for its credit so nothing is lost in the gap. Every recording mode has a byte ceiling; the arithmetic is in docs/REFERENCE.md.

Ships as `Serial_Decode.tspa`, installed from a USB key through the instrument's own **Manage Apps** screen. Written in TSP — Lua **5.0.2** embedded in the instrument firmware, which is why the sources avoid `#`, `%`, `string.gmatch` and bitwise operators throughout.

Which instruments

Only the DMM6500 has been tested. Everything else here is reasoning from the API surface, and it is labelled as such. The app needs three things from an instrument: a **digitizer** reachable as `dmm.digitize`, the **touchscreen app API** (`display.create` and friends, which is what makes it an app rather than a script), and **Lua 5.0.2**.

DMM6500	**Tested.** Two 17-hour soaks, 20 196 capture-and-decode cycles, firmware 1.7.17a
DAQ6510	Expected to run unmodified — same boards and firmware, the difference being the multi-channel scanner cards. Untested
DMM7510	Expected to run unmodified. Same `dmm.digitize` namespace and a faster digitizer. Untested, and see the `$Product` note below
2461 SMU	**Would need a port.** It has the hardware — Keithley call it a *Digitizing SourceMeter* with **dual 18-bit 1 MS/s digitizers** — but reaches it as `smu.`, not `dmm.`
2450 / 2460 / 2470 SMU	**No.** None of the three lists a digitize capability, and the 2470 manual says outright that digitized measurements are not a feature of the instrument

The SMU row is split because the series is not uniform, and the spec sheet hides that. Keithley's own 2400-series page carries "*1 MSamples/s digitized measurement speed*" in the family highlights and "*1 MS/s sampling*" in the spec banner — but the only model whose own section claims digitizers is the **2461** ("*dual 1 MS/s digitizers for fast sampling measurements*"). A family-level figure that belongs to one model reads as a series capability, which is how the 2450's decade-old reference documenting `smu.measure.*` with no digitize function at all, and a 2470 manual denying the feature outright, sit next to a 1 MS/s headline without any of them being wrong. **Check the model, not the series.**

So the 2461 is a genuine candidate and the others are not. The port it needs is small and bounded: the app's entire instrument surface is **14 `dmm.*` symbols** — `dmm.digitize.{read, count, samplerate, aperture, range, func, analogtrigger.mode, analogtrigger.edge.level, analogtrigger.edge.slope}`, `dmm.FUNC_DIGITIZE_VOLTAGE`, `dmm.MODE_EDGE`, `dmm.MODE_OFF`, `dmm.SLOPE_RISING`, `dmm.SLOPE_FALLING` — so one indirection table covers it. Two things are unverified and would decide how well it worked rather than whether it worked. Whether that digitizer exposes an **analog trigger**: without one the app falls back to the free-running path it already has. And what the **buffer** accepts rather than what the ADC samples — the same page quotes "*100 kS/s to buffer*" alongside the 1 MS/s figure, and this app reads captures out of the reading buffer, so the lower number would be the one that sets a baud ceiling. The DMM6500 has that same shape, measured: **~100 000 readings/s into the buffer** against a 1 MS/s digitizer.

The `.tspa` header decides where it will install, independently of whether the code would run. The shipped build declares `$Product: DMM6500, DAQ6510, DMM7510`, so all three are offered the app; the 2450-series SMUs are deliberately absent, because `$Product:` means "*can run the script without errors*" and an SMU would install it and then fail at the first capture. That is a false claim rather than an untested one.

One caveat on that header, recorded because it is cheap to check and annoying to debug: Keithley's app-header spec gives the permitted values as `2450, 2460, 2461, 2470, DMM6500, DAQ6510`, so **`DMM7510` is not in the documented set** — yet Keithley ship TSP apps of their own declaring DMM7510 support, which makes the published list stale or incomplete rather than authoritative. If a future firmware validates the field strictly and rejects the unknown token, the symptom would be an install failure, and `$Product:` is the first thing to revert.

Before releasing it to anyone

Three gates, each about ten times the cost of the one before it. Run them in order, so a cheap failure stops an expensive one. `docs/BENCH.md` describes the harness in full.

```
python3 tools/release_sweep.py --offline   #  ~1 min, no instruments
python3 tools/bench_smoke.py               #  11 min, both instruments
python3 tools/soak.py --hours 17 --suites formats,plan --skip-vectors v95,v96
```

The smoke gate is 11 minutes and covers the whole rate range. Four waveforms — the fox and a random payload, each in 8N1 and 7E1 — paired crosswise so one pair takes the 22 standard baud rates and the other the 21 drawn ones, and each rate family therefore faces both formats and both content classes. Then all 45 button presses. 86 cells, 9.5 min; presses, 1.9 min. Run it after any change and before starting anything long: a soak lap is three hours, and a harness defect found at minute 147 has cost an evening.

No version is tagged without at least 8 hours of soak, and the last two took 17. Not a guideline. A release sweep answers "does it pass?" once, which is the wrong question for anything that fails one run in ten — a single green sweep licenses an intermittent and a single red one reads as a regression nobody can reproduce. The soak reports a **failure rate per test point**, which is the number an intermittent actually has. A lap of 39 waveforms across 43 rates is 1,683 cells and about 2.9 hours measured, so 8 hours is the least that can distinguish "fails every lap" from "failed once" and 17 hours gives six laps — enough that a cell failing in all six is a property of the cell rather than of the evening.

Code changes do not get pushed without passing the smoke gate, and that is enforced rather than remembered. On a pass, `bench_smoke.py` writes a receipt holding a hash of `tsp/` and `tools/` as they were; a `pre-push` hook recomputes it and refuses if either tree has moved since. So the one-line fix made *after* the run invalidates the receipt, which is the case worth catching — that change is the least tested thing in the push. Install it with:

```
ln -sf ../../tools/hooks/pre-push .git/hooks/pre-push
```

Documentation-only pushes are unaffected; neither tree is hashed. When the bench is genuinely unavailable — mid-soak, say — `SMOKE_OVERRIDE="reason" git push` proceeds and prints the reason, so the exception is a decision on record rather than a silent bypass.

Every lap draws a different waveform order, different non-standard rates and a different wait before each capture, all from the lap number — so `--iteration N` rebuilds any lap exactly, without running the ones before it, and a failure replays offline in seconds. `docs/BENCH.md` has the commands.

```
python3 tools/release_sweep.py
```

That is the gate. It runs every check in one go and writes an auditable record to `out/release/<timestamp>/` — a `REPORT.md`, a `summary.json`, the full output of every stage, every front-panel screenshot taken, and the exact `.tspa` and `MANUAL.pdf` the results describe with their SHA-256s. It exits non-zero if any gate fails.

It needs a freshly power-cycled DMM6500. `sdec.start()` builds the display objects and the firmware does not fully return the pool, so the app refuses a second build in one power cycle. The sweep checks this up front and refuses with the remedy rather than failing forty minutes in. It also needs the SDG2122X

generator with the stimulus waveforms loaded (`bench_matrix.py --upload` puts them there) and only one client may hold the DMM's control socket at a time.

For a code change, the offline half runs in under ten seconds and needs no instruments:

```
python3 tools/release_sweep.py --offline
```

Stage	Checks
`lint` `parse`	Lua 5.0.2 incompatibilities; `luac -p` on every module
`vecrefs`	every waveform id named in `tools/` is in `MAP` or declared `RETIRED` — a deletion from `MAP` alone leaves references that fail where they are used, not where they are declared
`soakrand`	`mt19937.py` and `mt19937.lua` produce one sequence, checked against CPython's `random`, which is the same algorithm — plus the 5.0.2 syntax scan that decides whether the module can load on the instrument at all
`unit`	1,047 offline tests — decoder, UI, state machine, file paths, every visible ASCII glyph
`unit-cancel`	one-press recordings, both window sizes, and the flow-control loop running with no interaction
`stress`	hostile signals: never silently **wrong**, never **raises**
`unit-analog`	the bench cases at the app's own sample rates, swept over sampling phase, jitter, noise and where the capture window opens
`unit-phasesweep`	every stimulus vector × capture start × phase/jitter/noise, sharded across the cores — no raise, no result without a format, no wrong byte among those ERR calls trustworthy
`unit-seam`	a capture whose arb loop seam lands late must still be judged, and the narrower trim must still be required
`unit-sdgguard`	every route by which an out-of-spec waveform could reach the generator refuses it
`unit-loremgate`	the harness's own long-payload verdict — every clean run validated, flag count bounded
`tolerance`	recomputes the envelope table printed in the manual
`package` `archive`	rebuilds the `.tspa`, then builds both screens *from the archive* against a mock front end
`manual`	rebuilds every shipped PDF: `docs/MANUAL.pdf`, `docs/REFERENCE.pdf`, `README.pdf`
`hw-matrix`	on the bench, through the app's own Capture button: six frame formats, the standard rate ladder, a 1 kB non-repeating payload, three logic swings, twelve DC offsets
`hw-payloads`	fourteen distinct payloads covering every byte value 0–255, with the fox and a random vector also driven at 115 200 and 250 000
`hw-odd-rates`	nineteen **non-standard** baud rates — 900, 1500, 3600, 8123, 29127, 104857 …
`hw-panel`	every button in every state, with the panel grabbed **before and after each press** and differenced by region — including a one-press recording
`soakrand-dmm`	the third leg: the instrument's **own** Lua 5.0.2 runs the same generator and must produce the same words, floats, rejected draws and permutation

Stage	Checks
`hw-plan`	the seeded sweep on the bench: two waveforms across all 43 rates in a seeded order, with a seeded wait before every capture
`hw-break`	degenerate signals and contradictory settings — no signal, DC only, all-`0x00` / `0xFF` / `0x55`, a break, 60 mV of swing, 19 Vpp, rates past the ceiling, and six wrong forced settings. A refusal with a reason passes; confident garbage does not

`hw-panel` is the one worth understanding: it checks six things per press — that the handler does not raise, logs no instrument event, returns inside its latency budget, reports what it did, changed the state it was supposed to, *and that the panel actually shows it*. The last is a separate failure from the one before it, because the UI caches its writes: an app can be right while the operator reads something stale.

Layout

`tsp/`	the app. `serial_core` acquisition, `uart_decode` framing, `chunk_decode` resumable decode, `serial_ui` panel, `serial_app` orchestration
`tools/`	harnesses. `release_sweep.py` is the entry point; the rest are the individual authorities it calls
`docs/`	the manual, instrument references, panel mockups
`notes/`	bring-up log, findings, handoffs

`tsp/midi_decode.tsp` and `tsp/lin_decode.tsp` are complete and tested but **not shipped** in version 1 — the LIN checksum has never been checked against a real frame. Re-adding either is one line in `tools/package_tspa.py`; the app discovers what it has at runtime.

Bench

Three instruments over LAN: the DMM6500 under test, an SDG2122X generating the stimulus, an SDS1204X-E for verifying the stimulus is what it claims to be. `tools/instruments.py` holds the addresses and the hazards worth knowing — repeated large waveform uploads wedge the generator's LAN service, and calibration state is off limits on both.